

MP2: Frame Manager

Nithin Gowtham Saravanan

UIN: 337001954

CSCE611: Operating System

Assigned Tasks

Main: Completed.

Bonus Option 1: Marking Inaccessible memory (15MB-16MB)

Bonus Option 2: Testing Process Memory Pool

Bonus Option 3: Not applicable.

Bonus Option 4: Not applicable.

System Design

The objective of this machine problem is to implement a frame manager, which manages the allocation of frames (physical pages). The frame manager is responsible for the allocation and the release of frames, i.e, it needs to keep track of which pages are being used and which ones are free. Different parts of the memory will be used by different parts of the operating system, and so the frame manager will manage a collection of frame pools, which support get and release operations for frames. The frame manager allows for the creation of frame pool objects. Frame pools are each assigned a region of the physical memory. Callers request one or more contiguous frames by calling the appropriate frame pool's function get frames. The kernel, in file kernel.C, will set up two pools, the kernel frame pool and the process frame pool: Whenever the kernel needs frames, it gets them from the kernel frame pool, which is located between 2MB and 4MB. Non-kernel data is then allocated from the process frame pool, which manages memory above 4MB.

- A bitmap is used to track the state of each frame.
- Each frame is represented using 2 bits: Free, Used, or Head-of-Sequence (HoS).
- Contiguous sequences are allocated by marking the first frame as HoS and subsequent frames as Used.
- Free frames are merged by traversing the sequence from the HoS frame during deallocation.
- A global linked list of frame pools allows identifying the correct pool for any given frame.

Code Description

This implementation modifies the following files:

- `cont_frame_pool.H`
- `cont_frame_pool.C`
- `kernel.C`

cont_frame_pool.H: Class Definition This header defines the `ContFramePool` class, including:

- Bitmap representation using 2 bits per frame
- Frame state enum (`Free`, `Used`, `HoS`)
- function prototypes for allocating and de-allocating memory (`get_state` and `set_state`)
- creating linked list for each frame pool
- constructors and other function prototypes needed to handle the frame manager

cont_frame_pool.C: `get_state` It helps the allocator know which frames are available for allocation or need to be freed.

- To check the current status of a specific frame (`Free`, `Used`, or `HoS`) in the frame pool.
- Uses 2 bits per frame and 4 frames per byte

cont_frame_pool.C: `set_state` It marks frames as allocated, freed, or reserved during allocation, deallocation, or initialization.

- To update the status of a specific frame (`Free`, `Used`, or `HoS`) in the frame pool
- Clears the required bits and set the new state

cont_frame_pool.C: Constructor Initializes the frame pool:

- Allocates the bitmap in memory
- Marks all frames as `Free`
- Sets the first frame as `HoS` if needed for storing management information
- Initializes the pool list

cont_frame_pool.C: `get_frames` Allocates a contiguous sequence of frames:

- Scans the bitmap for a run of free frames of requested length
- Marks first frame as `HoS`, remaining as `Used`
- Returns absolute frame number of the first frame
- If no contiguous block is found, it asserts

cont_frame_pool.C: `mark_inaccessible` Marks a sequence of frames as allocated without searching:

- For memory range 15MB-16MB
- Sets first frame as `HoS`, remaining as `Used`

cont.frame_pool.C: release_frames Releases a contiguous sequence:

- Identifies the correct pool via the global pool list
- Verifies the first frame is HoS
- Frees all subsequent Used frames until the next Free or HoS frame
- If the requested pool is not available for the given frame, it asserts

cont.frame_pool.C: needed_info_frames Calculates the number of frames required for bitmap:

- Uses 2 bits per frame
- Rounds up to the nearest frame

kernel.C: Holds the source code for the kernel with the test function

- Uncommented the code for process pool and mark inaccessible memory
- invoking the test_memory function with the process pool data

Bonus Option 1:

Marking Inaccessible Memory

kernel.c: mark_inaccessible : Implementation is done as part of the requirements (i.e., first frame is set to HoS and the consecutive frames are set to Used). Testing is performed by calling the function, and debug comments are added to display in the output.

Bonus Option 2:

Testing Process memory pool

kernel.c: process_mem_pool : Implementation is done as part of the requirements. Testing is performed by calling the function test_memory but this time with process pool class for the same test cases as kernel pool. Debug comments are added to display in the output.

Testing

Testing involved

- Allocating and freeing sequences of various lengths
- Marking frames as inaccessible and ensuring allocation avoids them
- Verifying nFreeFrames is updated correctly
- Confirming that the returned frame numbers are correct absolute addresses

TestProcedure

1. Initialization of Kernel Memory Pool:

- (a) Configures the bitmap frame.
- (b) Traverses through all frames within the specified memory range (3MB–4MB) and sets them as Free.
- (c) Initializes the linked list for pool traversal for the kernel pool.

2. Initialization of Process Memory Pool:

- (a) Calls `needed_info_frames` to calculate the number of frames required to store bitmap information for the process pool.
- (b) Uses `get_frames` to allocate the required frames contiguously to store bitmap info. In this case, 1 frame is enough.
- (c) Configures the bitmap frame.
- (d) Traverses through all frames within the specified memory range (5MB–32MB) and sets them as Free.
- (e) Initializes the linked list for pool traversal for the process memory pool.

3. Marking Inaccessible Memory:

- (a) The range 15MB–16MB in the process pool is marked as inaccessible.
- (b) Sets the starting frame of 15MB as HoS (Head of Segment) and the remaining 511 frames (up to 16MB) as `Used`.

4. Testing Kernel Pool:

- (a) Calls the `test_memory` function.
- (b) For each requested allocation, the pool traverses from the beginning to find contiguous free frames.
- (c) Marks the first frame as HoS and the remaining frames as `Used`, returning the base frame number.
- (d) Performs 32 allocations as per the test software.
- (e) Calls `release_frames` with the allocated frame number.
- (f) If the frame corresponds to HoS, it frees that frame and all successive frames until the next HoS.
- (g) All frames are released in this manner.
- (h) Debug output shows the allocation and release process.
- (i) Note: The last frame freed is frame number 2, as the first frame is reserved for the bitmap (4KB = 4096 bytes, 2 bits per frame).

5. Testing Process Memory Pool:

- (a) Executed in the same manner as the kernel pool, using `test_memory`.
- (b) Allocations and releases follow the same procedure with proper debug output.

All tests were performed using the provided console print utilities and assertion checks.